```
    void lock()
    {
        while(flag.test_and_set(std::memory_order_acquire));
    }
    void unlock()
    {
        flag.clear(std::memory_order_release);
    }
};
```

This code doesn't call any blocking functions; `lock()` just keeps looping until the call to `test_and_set()` returns `false`. This is why it gets the name *spin lock*—the code "spins" around the loop. Anyway, there are no blocking calls, so any code that uses this mutex to protect shared data is consequently *nonblocking*. It's not *lock-free*, though. It's still a mutex and can still be locked by only one thread at a time. Let's look at the definition of *lock-free* so you can see what kinds of data structures *are* covered.

7.1.2. Lock-free data structures

For a data structure to qualify as lock-free, more than one thread must be able to access the data structure concurrently. They don't have to be able to do the same operations; a lock-free queue might allow one thread to push and one to pop but break if two threads try to push new items at the same time. Not only that, but if one of the threads accessing the data structure is suspended by the scheduler midway through its operation, the other threads must still be able to complete their operations without waiting for the suspended thread.

Algorithms that use compare/exchange operations on the data structure often have loops in them. The reason for using a compare/exchange operation is that another thread might have modified the data in the meantime, in which case the code will need to redo part of its operation before trying the compare/exchange again. Such code can still be lock-free if the compare/exchange would eventually succeed if the other threads were suspended. If it wouldn't, you'd essentially have a spin lock, which is nonblocking but not lock-free.